



CS101 / 题库 (包括计概、数算题目)

题目 排名 状态 提问

#47383826提交状态

查看 提交 统计 提问

状态: Accepted

源代码

```
def dfs(matrix, row, col, visited):
    if row < 0 or row >= len(matrix) or col < 0 or col >= len(matrix[0]) or matrix[row][col] != 'W' or visited[row][col]:
        return 0

    visited[row][col] = True
    size = 1

    for dr in [-1, 0, 1]:
        for dc in [-1, 0, 1]:
            size += dfs(matrix, row + dr, col + dc, visited)

    return size

def max_connected_area(matrix):
    max_area = 0
    visited = [[False for _ in range(len(matrix[0]))] for _ in range(len(matrix))]

    for row in range(len(matrix)):
        for col in range(len(matrix[0])):
            if matrix[row][col] == 'W' and not visited[row][col]:
                area = dfs(matrix, row, col, visited)
                max_area = max(max_area, area)

    return max_area
```

基本信息

#: 47383826
题目: 18160
提交人: cislobog_harute
内存: 3740kB
时间: 133ms
语言: Python3
提交时间: 2024-11-25 11:12:20



CS101 / 题库 (包括计概、数算题目)

题目

排名

状态

提问

#47383964提交状态

查看

提交

统计

提问

状态: Accepted

源代码

```
import heapq
def bfs(x, y):
    d=[[-1,0],[1,0],[0,1],[0,-1]]
    queue=[]
    heapq.heappush(queue,[0,x,y])
    check=set()
    check.add((x,y))
    while queue:
        step,x,y=map(int,heapq.heappop(queue))
        if martix[x][y]==1:
            return step
        for i in range(4):
            dx,dy=x+d[i][0],y+d[i][1]
            if martix[dx][dy]!=2 and (dx,dy) not in check:
                heapq.heappush(queue,[step+1,dx,dy])
                check.add((dx,dy))
    return "NO"

m,n=map(int,input().split())
martix=[[2]*(n+2)]+[[2]+list(map(int,input().split()))+2] for i in range(m)
print(bfs(1,1))
```

基本信息

#: 47383964

题目: 19930

提交人: cislobog_harute

内存: 3700kB

时间: 33ms

语言: Python3

提交时间: 2024-11-25 11:24:58



CS101 / 题库 (包括计概、数算题目)

题目

排名

状态

提问

#47408258提交状态

查看

提交

统计

提问

状态: Accepted

源代码

```
maxn = 10;
sx = [-2, -1, 1, 2, 2, 1, -1, -2]
sy = [1, 2, 2, 1, -1, -2, -2, -1]

ans = 0;

def Dfs(dep: int, x: int, y: int):

    if n * m == dep:
        global ans
        ans += 1
        return

    for r in range(8):
        s = x + sx[r]
        t = y + sy[r]
        if chess[s][t] == False and 0 <= s < n and 0 <= t < m:
            chess[s][t] = True
            Dfs(dep + 1, s, t)
            chess[s][t] = False;

for _ in range(int(input())):
    n, m, x, y = map(int, input().split())
```

基本信息

#: 47408258

题目: 04123

提交人: cislobog_harute

内存: 3700kB

时间: 3356ms

语言: Python3

提交时间: 2024-11-26 16:04:11

提高篇 (2) —— 搜索专题

深度优先搜索 (DFS)

- 迷宫可行路径数
- 指定步数的迷宫问题
- 矩阵最大权值
- 矩阵最大权值路径
- 迷宫最大权值

题目 题解

矩阵最大权值路径

通过数 972 提交数 2567 难度 中等 显示标签 ☆

题目描述

现有一个 $n * m$ 大小的矩阵，矩阵中的每个元素表示该位置的权值。现需要从矩阵左上角出发到达右下角，每次移动只能向上下左右移动一格（不允许移动到曾经经过的位置）。假设左上角坐标是 $(1, 1)$ ，行数增加的方向为 x 增长的方向，列数增加的方向为 y 增长的方向。求最后到达右下角时路径上所有位置的权值之和最大的路径。

输入描述

第一行两个整数 n, m ($2 \leq n \leq 5, 2 \leq m \leq 5$)，分别表示矩阵的行数和列数；
接下来 n 行，每行 m 个整数 ($-100 \leq \text{整数} \leq 100$)，表示矩阵每个位置的权值。

输出描述

从左上角的坐标开始，输出若干行（每行两个整数，表示一个坐标），直到右下角的坐标。

数据保证权值之和最大的路径存在且唯一。

代码书写

```
3  if x == n - 1 and y == m - 1:
4     if now_value > max_value:
5         max_value = now_value
6         opt_path = temp_path[:]
7     return
8
9  visited[x][y] = True
10
11  for dx, dy in directions:
12      next_x, next_y = x + dx, y + dy
13      if 0 <= next_x < n and 0 <= next_y < m and not visited[next_x][next_y]:
14          next_value = now_value + maze[next_x][next_y]
15          temp_path.append((next_x, next_y))
16          dfs(next_x, next_y, next_value)
17          temp_path.pop()
18
19  visited[x][y] = False
20
21
22  n, m = map(int, input().split())
```

测试输入

提交结果

历史提交

完美通过

[查看题解](#)

100% 数据通过测试

运行时长: 0 ms

收起面板

运行

提交

题目描述 题解 通过 x 提交记录

全部提交记录

通过

Quirky Greider135 提交于 2024.11.26 17:19

官方题解 写题解



「感恩季」限时福利！
买 1 得 2 同时可享力扣 365 天 Plus 会员 + 热门平台会员权益。

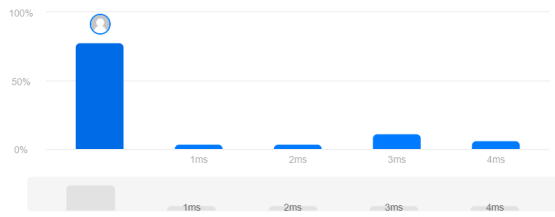
执行用时分布

0 ms | 击败 100.00%

复杂度分析

消耗内存分布

16.93 MB | 击败 5.01%



代码 | Python3

代码

Python3 智能模式

```
1 class Solution:
2     def uniquePaths(self, m: int, n: int) -> int:
3         dp = [[1]*n] + [[1]+[0] * (n-1) for _ in range(m-1)]
4         #print(dp)
5         for i in range(1, m):
6             for j in range(1, n):
7                 dp[i][j] = dp[i-1][j] + dp[i][j-1]
8         return dp[-1][-1]
9
10
```

已存储

行 9, 列 1

测试用例 测试结果

通过 执行用时: 0 ms

Case 1 Case 2

输入

m =
3

n =
7

输出

- 提高篇 (2) —— 搜索专题
- 综合练习精选
- 受到祝福的平方

题目题解

返回题解列表

Python题解：受到祝福的平方

AI 题解Python

📌 本题解由 AI 大模型自动生成，可能存在幻觉现象，部分细节请注意辨别

思路：深度优先搜索 (DFS)

本题的核心在于判断一个给定的正整数 A 是否可以被分割成若干个正整数的平方数。由于 A 的最大值为 10^9 ，其数字长度最多为 9 位，因此我们可以通过深度优先搜索 (DFS) 来暴力枚举所有可能的分割方式，并判断是否满足条件。

整体思路：

- 预处理平方数：**首先，我们预处理出所有可能的平方数，并将其存储在一个集合中。由于 A 的最大值为 10^9 ，我们需要计算出所有小于等于 10^9 的平方数。
- 数字分割：**将输入的数字 A 转换为一个数字列表，从高位到低位进行分割。
- DFS 搜索：**使用 DFS 从数字的最高位开始，尝试将数字分割成若干部分，并检查每一部分是否为某个正整数的平方数。如果能够成功分割，则返回 Yes，否则返回 No。

具体过程细节：

- 预处理平方数：**我们通过一个循环生成所有小于等于 10^9 的平方数，并将它们存储在一个集合中。这样可以在后续的 DFS 过程中快速判断某个数字是否

代码书写

⚙️ Python

```
1 def is_blessed_id(A):
2
3     squares = set()
4     i = 1
5     while i * i <= 10 ** 9:
6         squares.add(i * i)
7         i += 1
8
9     digits = list(map(int, str(A)))
10
11     def dfs(idx):
12         if idx == len(digits):
13             return True
14
15         num = 0
16         for i in range(idx, len(digits)):
17             num = num * 10 + digits[i]
18             if num in squares:
19                 if dfs(i + 1):
20                     return True
```

测试输入提交结果历史提交

完美通过

查看题解

100% 数据通过测试

运行时长: 0 ms

收起面板

运行提交